IF3170 - INTELIGENSI ARTIFISIAL

Tugas Besar I

Implementasi Penyusunan Paket Container Menggunakan Local Search



Disusun Oleh

Mikhael Benrael Tampubolon - 13524009
Marcel Luther Sitorus - 13524063
Richard Samuel Simanullang - 13524112

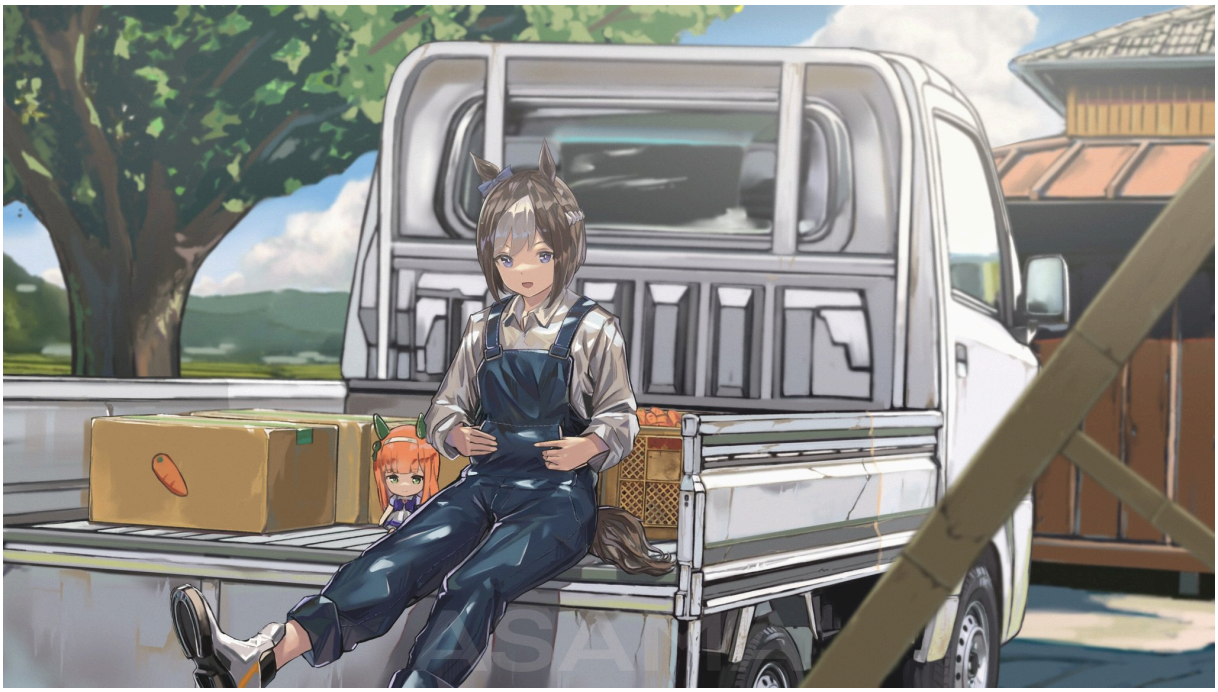
Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

Deskripsi Persoalan



Gambar 1. Special Week, driver truk handal (Sumber: 真夏の银堡垒🐴C108土曜日 東地区 ホ-10a on X)

Keluarga **Mejiro** sedang berencana mengekspansi bisnisnya ke bidang logistik darat dan ingin melakukan uji coba berskala kecil terlebih dahulu untuk torasen. Sebagai teman yang baik, **Mejiro McQueen** menawarkan posisi driver truk ujicoba kepada **Special Week**, karena sudah sering menyetir truk di kampung halamannya di Hokkaido. Usaha baru ini bernama **Mejiro Express** dengan slogan “Lebih kilat dari Umamusume”.

Karena **Special Week** sibuk berlatih untuk memenangkan lomba-lombanya, dia tidak bisa membuang waktu berlama-lama untuk menentukan pengaturan pengiriman yang optimal. Oleh karena itu, dia meminta kalian, toreena-toreena yang pintar dalam keinformatikawan, untuk membuat suatu program yang cerdas untuk menolongnya.

Dalam menjalankan **Mejiro Express**, **Special Week** perlu menentukan paket-paket yang akan dimuat ke dalam truk peti kemas agar nilai total paket yang diangkut dapat dimaksimalkan. Setiap paket memiliki dimensi, nilai, berat, sifat mudah pecah, dan

perkiraan waktu kedatangan (*ETA)*. Sementara itu, truk memiliki dimensi peti kemas dan kapasitas angkut maksimum yang harus diperhatikan dalam penyusunan paket.

Penyusunan paket tidak dapat dilakukan secara sembarangan. Terdapat beberapa aturan yang perlu diperhatikan

1. Setiap paket harus ditempatkan di dalam atau di luar peti kemas secara utuh, tanpa menembus dinding peti kemas.
2. Paket tidak boleh melayang
3. Barang mudah pecah tidak boleh berada di bawah paket lain maupun jadi penopang paket lain
4. Total berat paket yang diangkut tidak boleh melebihi kapasitas truk
5. Paket dapat diputar untuk menyesuaikan orientasinya dengan ruang yang tersedia.

Untuk membantu **Special Week**, **Pengembara Kakas** mengembangkan program yang menerapkan algoritma *local search*. Program ini memulai pencarian dari konfigurasi paket yang dibangkitkan secara acak, kemudian mengeksplorasi konfigurasi lain melalui operasi pemindahan posisi (*Move),* rotasi orientasi (*Rotate),* dan pertukaran posisi antar paket (*Swap)*. Setiap konfigurasi dievaluasi menggunakan fungsi objektif untuk memaksimalkan nilai paket yang diangkut dan mempertimbangkan penalti atas pelanggaran batasan.

Terdapat beberapa algoritma yang diterapkan dan dibandingkan, yaitu varian Hill-Climbing, Simulated Annealing, dan Genetic Algorithm. Melalui eksperimen terhadap algoritma-algoritma tersebut, diharapkan dapat diperoleh konfigurasi penyusunan paket yang memberikan nilai objektif tinggi serta diketahui karakteristik masing-masing algoritma.

Pembahasan

I. Representasi State dan Fungsi Objektif

State direpresentasikan menggunakan kelas State yang menyimpan objek truk (Truck), daftar paket (Package), serta daftar representasi bentuk (PackRepresentation) yang mencatat posisi koordinat (x, y, z) dan orientasi rotasi 3D setiap paket.

Fungsi objektif dasar dievaluasi melalui fungsi score(state) pada src/backend/app/utils/objective.py yang menghitung total nilai (*value*) seluruh paket berstatus INSIDE dikurangi penalti pelanggaran (*violations*):

```python
def score(state: State) -> int:
    return objective(state)-penalty_weight(state)*cnt_violations(state).total
```

Aksi pencarian tetangga (*neighbors*) didefinisikan di src/backend/app/utils/neighbors.py melalui fungsi all_neighbors(state) dan random_neighbor(state) yang mencakup tiga operasi dasar: Move (pindah koordinat), Rotate (rotasi sumbu x, y, z), dan Swap (tukar posisi antar dua paket).

```python
def all_neighbors(state: State) -> Iterator[State]:
    n = len(state.packages)
    for i in range(n):
        for j in range(i + 1, n):
            if state.shape_pack_models[i].position.get_xyz() != state.shape_pack_models[j].position.get_xyz():
                yield swap(state, i, j)
    for i in range(n):
        for axis in ROTATION_SWAPS:
            yield rotate(state, i, axis)
        current = state.shape_pack_models[i].position.get_xyz()
        for position in all_pos(state.truck, package_size(state, i)):
            if position.get_xyz() != current:
                yield move(state, i, position)
```

```python
def random_neighbor(state: State, rng=random) -> State:
    n = len(state.packages)
    kind = rng.choice(("swap", "move", "rotate") if n >= 2 else ("move", "rotate"))
    i = rng.randrange(n)
    if kind == "swap":
        j = rng.choice([k for k in range(n) if k != i])
        return swap(state, i, j)
    if kind == "rotate":
        return rotate(state, i, rng.choice(tuple(ROTATION_SWAPS)))
    return move(state, i, random_pos(state.truck, package_size(state, i), rng))
```

```python
# gerakan dasar
def swap(state: State, i: int, j: int) -> State:
    new_state = copy_state(state)
    a, b = new_state.shape_pack_models[i], new_state.shape_pack_models[j]
    a_position = a.position
    a.set_position(b.position)
    b.set_position(a_position)
    return new_state

def move(state: State, i: int, position: Position) -> State:
    new_state = copy_state(state)
    new_state.shape_pack_models[i].set_position(position)
    return new_state

def rotate(state: State, i: int, axis: str) -> State:
    new_state = copy_state(state)
    rep = new_state.shape_pack_models[i]
    rep.set_orientation(rotated(rep.orientation, axis))
    return new_state


# posisi yang mungkin di truk
def all_pos(truck: Truck, size: tuple[int, int, int]) -> list[Position]:
    limit = truck.dimension.get_wlh()
    ranges = [range(limit[k] - size[k] + 1) for k in range(3)]
    return [Position(*point) for point in product(*ranges)] + [outside_position(truck)]

# random posisi di truk
def random_pos(truck: Truck, size: tuple[int, int, int], rng=random) -> Position:
    room = [limit - s for limit, s in zip(truck.dimension.get_wlh(), size)]
    if all(r >= 0 for r in room) and rng.random() < 0.5:
        return Position(*(rng.randint(0, r) for r in room))
    return outside_position(truck)
```

II. Aturan Eksperimen

Truck: W×L×H = 10×20×5 (volume 1000), kapasitas berat 500. 12 paket, total value 2050, total berat 775, total volume 1445. Tidak semua paket bisa dimuat. Berikut detail tiap paket

Paket	W×L×H	Value	Berat	Fragile
PKG-1	4×6×3	120	40	-
PKG-2	5×8×4	220	90	-

PKG-3	3×4×1	60	5	-
PKG-4	6×10×3	260	110	ya
PKG-5	4×4×4	90	30	-
PKG-6	2×5×2	45	12	ya
PKG-7	8×12×5	520	200	-
PKG-8	5×5×2	70	25	-
PKG-9	7×9×4	310	140	-
PKG-10	3×6×3	80	20	ya
PKG-11	6×6×5	200	85	-
PKG-12	4×7×2	75	18	-

State awal. Setiap algoritma dijalankan 3 kali dengan seed 42, 43, 44 (run 1, 2, 3). Seed yang sama menghasilkan state awal yang sama, sehingga run ke-k di setiap algoritma berangkat dari state awal yang sama (S1, S2, S3) dan hasilnya bisa dibandingkan langsung. Paket yang tidak disebut berada di luar truk. Notasi: PKG (x,y,z) ukuran-terorientasi.

State	**Seed**	**Objective**	**Paket di dalam truk**
S1	42	−15633	PKG-1 (4,3,0) 3×6×4; PKG-3 (8,2,2) 1×4×3; PKG-5 (1,7,0) 4×4×4; PKG-6 (8,13,0) 2×2×5; PKG-8 (0,5,0) 2×5×5; PKG-9 (0,12,1) 9×7×4; PKG-10 (1,5,1)

			3×6×3
S2	43	−11846	PKG-1 (6,2,1) 4×6×3; PKG-3 (7,0,4) 3×4×1; PKG-5 (6,13,1) 4×4×4; PKG-6 (1,3,0) 2×2×5; PKG-8 (3,4,3) 5×5×2; PKG-12 (0,5,0) 7×2×4
S3	44	−5683	PKG-3 (4,0,0) 3×1×4; PKG-5 (4,9,1) 4×4×4; PKG-6 (5,11,0) 2×2×5; PKG-11 (2,1,0) 6×6×5; PKG-12 (0,7,3) 4×7×2

III. Hill-Climbing : Steepest Ascent

Hill-Climbing merupakan pencarian solusi yang terus menerus berpindah ke state yang valuenya lebih besar. Diimplementasikan dengan menggunakan loop utama berikut :

```
        while True:
            best_neighbor = None
            best_neighbor_val = float('-inf')

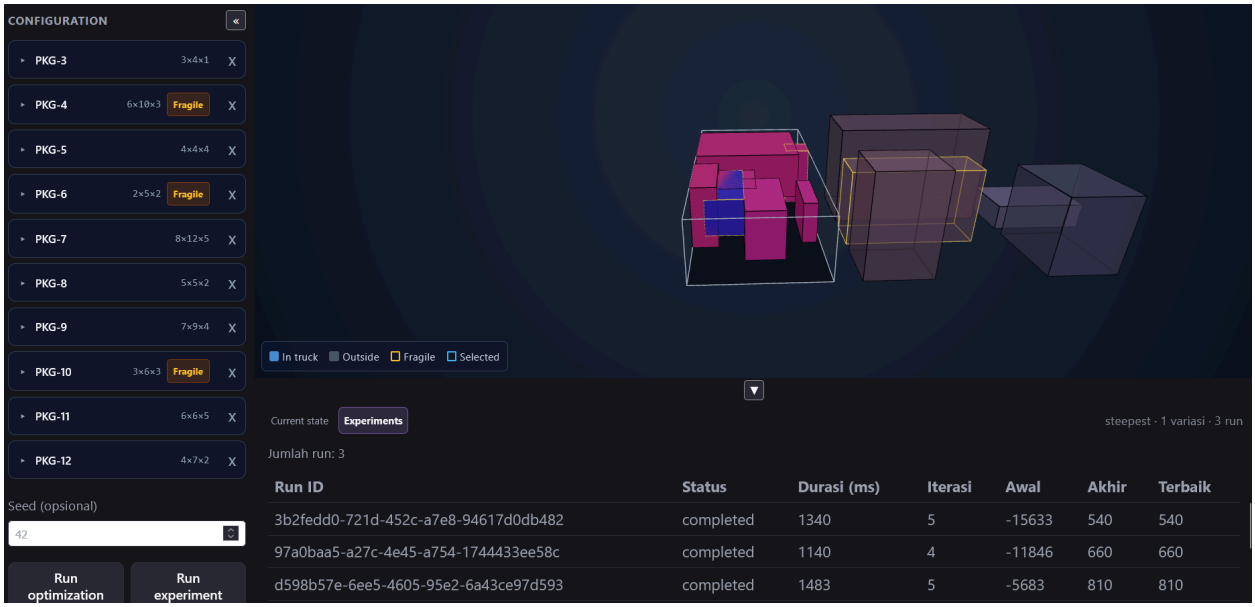
            for succ in all_neighbors(curr):
                succ_val = obj_fn(succ)
                if succ_val > best_neighbor_val:
                    best_neighbor = succ
                    best_neighbor_val = succ_val
```

Di setiap iterasi, semua neighbor dievaluasi dan yang nilainya paling tinggi dicari. Kondisi berhenti ketika:

- best_neighbor is None: tidak ada neighbor sama sekali (misalnya tidak ada paket).
- best_neighbor_val <= curr_val: neighbor terbaik tidak lebih baik dari state sekarang.

Hasil Eksperimen:

Run	**State awal**	**State akhir (paket di dalam truk)**	**Objective akhir**	**Iterasi hingga berhenti**	**Durasi**
1	S1 (−15633)	PKG-1 (4,3,0) 3×6×4; PKG-3 (0,10,0) 1×4×3; PKG-5 (0,0,0) 4×4×4; PKG-6 (8,13,0) 2×2×5; PKG-8 (0,5,0) 2×5×5; PKG-10 (7,6,0) 3×6×3; PKG-12 (0,10,3) 7×4×2	**540**	5	922 ms
2	S2 (−11846)	PKG-1 (0,7,0) 4×6×3; PKG-3 (4,10,0) 3×4×1; PKG-5 (6,13,1) 4×4×4; PKG-6 (1,3,0) 2×2×5; PKG-8 (0,7,3) 5×5×2; PKG-11 (0,14,0) 6×6×5; PKG-12 (0,5,0) 7×2×4	**660**	4	801 ms
3	S3 (−5683)	PKG-1 (4,7,0) 6×3×4; PKG-2 (5,11,0) 4×8×5; PKG-3 (4,0,0) 3×1×4; PKG-5 (0,14,0) 4×4×4; PKG-6 (0,0,0) 2×2×5; PKG-11 (2,1,0) 6×6×5; PKG-12 (0,7,0) 4×7×2	**810**	5	925 ms



Analisis Eksperimen:

Ketiga run berhenti setelah 4–5 iterasi dengan pola yang sama. Langkah-langkah awal menaikkan objective ±2000–6000 poin karena setiap langkah menghapus satu atau lebih pelanggaran (penalti 2051 per pelanggaran). Setelah state valid tercapai, setiap langkah hanya menambah value satu paket (misalnya +75, +200, +120). Sesudah itu tidak ada lagi neighbor yang lebih baik, jadi algoritma berhenti di local optimum.

Hasil akhirnya 540–810, atau 41–61% dari batas atas 1330. Penyebabnya, local optimum di masalah ini sangat banyak. Setelah beberapa paket ditempatkan, ruang kosong yang tersisa terpecah-pecah. Memasukkan paket besar (PKG-7, PKG-4, PKG-9) ke celah itu butuh beberapa langkah sekaligus, misalnya menggeser paket lain dulu. Steepest ascent tidak bisa melakukannya karena setiap langkah harus menaikkan objective. Contohnya run 1: berat yang termuat baru 150 dari 500, tetapi tidak ada satu langkah pun yang bisa menambah paket.

IV. Hill-Climbing : Sideways

Varian Hill-Climbing ini memperbolehkan perubahan neighbor menjadi successor jika valuenya sama dengan value neighbornya. Ambil neighbor terbaik, Berhenti kalau tidak ada neighbor, atau neighbor terbaik lebih buruk dari state sekarang, artinya curr adalah local maximum yang sebenarnya.Perhatikan bedanya

dengan steepest ascent: di sana syaratnya <=, di sini <. Nilai yang sama tidak langsung membuat berhenti.

1. Syarat pindah

```
if (neigh is None or neigh_val < curr_val):
    break
if (neigh_val == curr_val):
    if (self.sideways_moves >= self.max_sideways):
        break
    self.sideways_moves += 1
curr = neigh
curr_val = neigh_val
```

2. Pilih neighbor terbaik untuk jadi successor

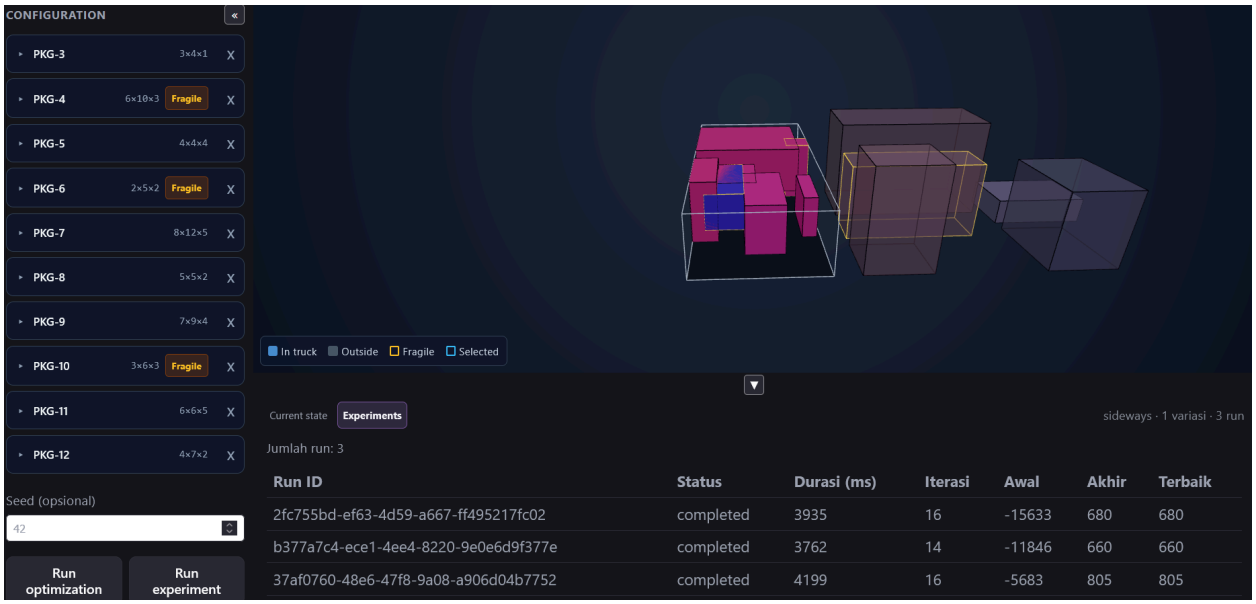
```
if (neighbor_val is None or succ_val > neighbor_val):
    neighbor = succ
    neighbor_val = succ_val
    ties = 1
elif (succ_val == neighbor_val):
    ties += 1
    if (self.rng.randrange(ties) == 0):
        neighbor = succ
```

Jika terdapat nilai neighbor yang seri, dipilih secara random.

Hasil Eksperimen:

Run	State awal	State akhir (paket di dalam truk)	Objective akhir	Iterasi hingga berhenti	Sideways move	Durasi
1	S1 (−15633)	PKG-1 (2,1,0) 3×6×4; PKG-2 (2,9,0) 8×4×5; PKG-3 (1,12,0) 1×4×3; PKG-5 (6,1,0) 4×4×4; PKG-6 (8,13,0) 2×2×5; PKG-8 (0,5,0) 2×5×5;	**680**	16	10	2698 ms

		PKG-12 (1,15,3) 7×4×2				
2	S2 (−11846)	PKG-1 (2,6,0) 4×6×3; PKG-3 (7,16,0) 3×4×1; PKG-5 (6,13,1) 4×4×4; PKG-6 (0,7,0) 2×2×5; PKG-8 (4,8,3) 5×5×2; PKG-11 (0,14,0) 6×6×5; PKG-12 (3,2,0) 7×2×4	**660**	14	10	2442 ms
3	S3 (−5683)	PKG-1 (4,7,0) 6×3×4; PKG-2 (1,10,0) 4×8×5; PKG-3 (2,18,0) 3×1×4; PKG-5 (6,15,0) 4×4×4; PKG-6 (0,7,0) 2×2×5; PKG-8 (5,10,0) 5×5×2; PKG-11 (2,1,0) 6×6×5	**805**	16	10	2738 ms



Analisis Eksperimen:

Di ketiga run, pencarian berhenti karena batas 10 sideways move tercapai, bukan karena tidak ada lagi neighbor yang seri. Artinya local optimum di masalah ini hampir selalu berupa plateau: banyak neighbor bernilai sama, misalnya paket di luar truk dipindah ke posisi luar lain, atau paket di dalam digeser ke posisi kosong lain tanpa mengubah validitas.

Sideways move membantu ketika *plateau* tersebut ternyata *shoulder*. Run 1 adalah contoh paling jelas, steepest ascent berhenti di 540, sedangkan sideways move terus bergerak datar 9 langkah di 540 sampai pergeseran paket membuka ruang untuk PKG-2, lalu naik ke 680. Di run 2 dan 3, sideways move hanya sekali dipakai di tengah pendakian, lalu sisa 9 sideways move habis di puncak akhir tanpa menemukan jalan naik.

V. Hill-Climbing : Stochastic

Varian Hill-Climbing ini memilih satu neighbor secara acak pada setiap iterasi, bukan mengevaluasi seluruh neighbor untuk mencari yang terbaik. Neighbor hanya menjadi successor jika valuenya lebih tinggi daripada state saat ini. Jika value sama atau lebih rendah, state saat ini akan dipertahankan. Proses pencarian dilakukan hingga mencapai jumlah iterasi maksimum yang ditentukan.

1. Pemilihan neighbor secara acak

```
neigh = random_neighbor(curr, self.rng)
neigh_val = obj_fn(neigh)
```

Pada setiap iterasi, *random_neighbor()* memilih satu neighbor secara acak, kemudian *obj_fn()* menghitung valuenya.

2. Syarat perpindahan ke successor

```
if (neigh_val > curr_val):
    curr = neigh
    curr_val = neigh_val
```

Neighbor menjadi successor hanya jika valuenya lebih tinggi daripada state saat ini. Jika valuenya sama atau lebih rendah, perpindahan ditolak

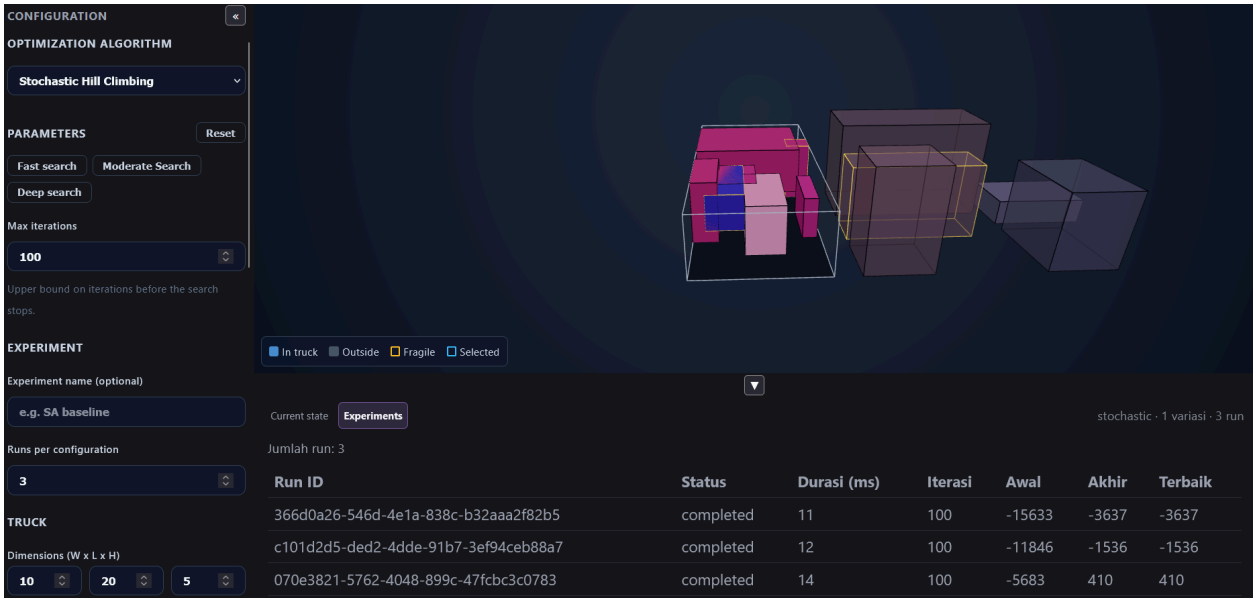
3. Batas iterasi

```
for _ in range(self.max_iterations):
```

Berbeda dengan Steepest Ascent dan Sideways yang berhenti berdasarkan kondisi neighbor, Stochastic menjalankan pencarian hingga mencapai *max_iterations.* Algoritma akan tetap mencoba neighbor baru meskipun beberapa iterasi berturut-turut tidak menghasilkan peningkatan value.

Hasil Eksperimen:

Run	**State awal**	**State akhir (paket di dalam truk)**	**Objective akhir**	**Iterasi hingga berhenti**	**Durasi**
1	S1 (−15633)	PKG-1 (4,3,0) 3×6×4; PKG-3 (0,5,0) 1×4×3; PKG-5 (4,10,0) 4×4×4; PKG-6 (8,13,0) 2×2×5; PKG-8 (8,15,0) 2×5×5; PKG-10 (1,5,1) 3×6×3; PKG-2 sebagian keluar truk *(masih 2 pelanggaran)*	**−3637**	100	7 ms
2	S2 (−11846)	PKG-1 (6,2,1) 4×6×3; PKG-3 (7,0,4) 3×4×1; PKG-5 (3,12,0) 4×4×4; PKG-6 (1,3,0) 2×2×5; PKG-11 (0,5,0) 6×6×5 *(masih 1 pelanggaran)*	**−1536**	100	7 ms
3	S3 (−5683)	PKG-3 (4,0,0) 3×1×4; PKG-4 (2,1,0) 6×10×3; PKG-5 (5,11,0) 4×4×4	**410**	100	6 ms



Analisis Eksperimen:

Stochastic hill climbing paling cepat di antara varian hill climbing (~7 ms, sekitar 125 kali lebih cepat dari steepest ascent), karena setiap iterasi hanya mengevaluasi satu neighbor, bukan ±3.500. Kekurangannya, satu neighbor acak jarang meningkatkan value, kebanyakan neighbor memindahkan paket ke posisi yang menimbulkan tabrakan atau melayang. Pada riwayat objective terlihat algoritma bisa bertahan puluhan iterasi di nilai yang sama (misalnya 37 dan 46 iterasi) karena semua neighbor acak yang dicoba ditolak.

VI. Hill-Climbing : Random Restart

Varian Hill-Climbing ini melakukan serangkaian pencarian (dengan *initial state* acak, hingga tujuan ditemukan) atau jumlah restar maksimum tercapai. Namun jika pencarian *stuck* pada suatu *local optimum*, maka akan dilakukan pencarian baru dari konfigurasi paket yang dibangkitkan secara acak

1. Membangkitkan initial state baru

```
for restart in range(self.max_restart + 1):
            if (restart > 0):
                state = random_state(init_state.truck,
init_state.packages, self.rng)
```

Pada restart pertama, algoritma menggunakan initial state yang diberikan. Pada restart berikutnya, *random_state()* membangkitkan konfigurasi paket baru untuk memulai pencarian dari titik yang berbeda

2. Menjalakan Hill-Climbing pada setiap restart

```
climber = HillClimbingSideways(max_sideways=0, rng=self.rng)
{iterasi...}
      res = climber.run(state, obj_fn)
```

Setiap konfigurasi dijalankan menggunakan Sideways, sehingga perpindahan ke neighbor dengan value sama tidak diperbolehkan. Hasil setiap pencarian kemudian dibandingkan untuk menyimpan solusi dengan value tertinggi

3. Kondisi Berhenti

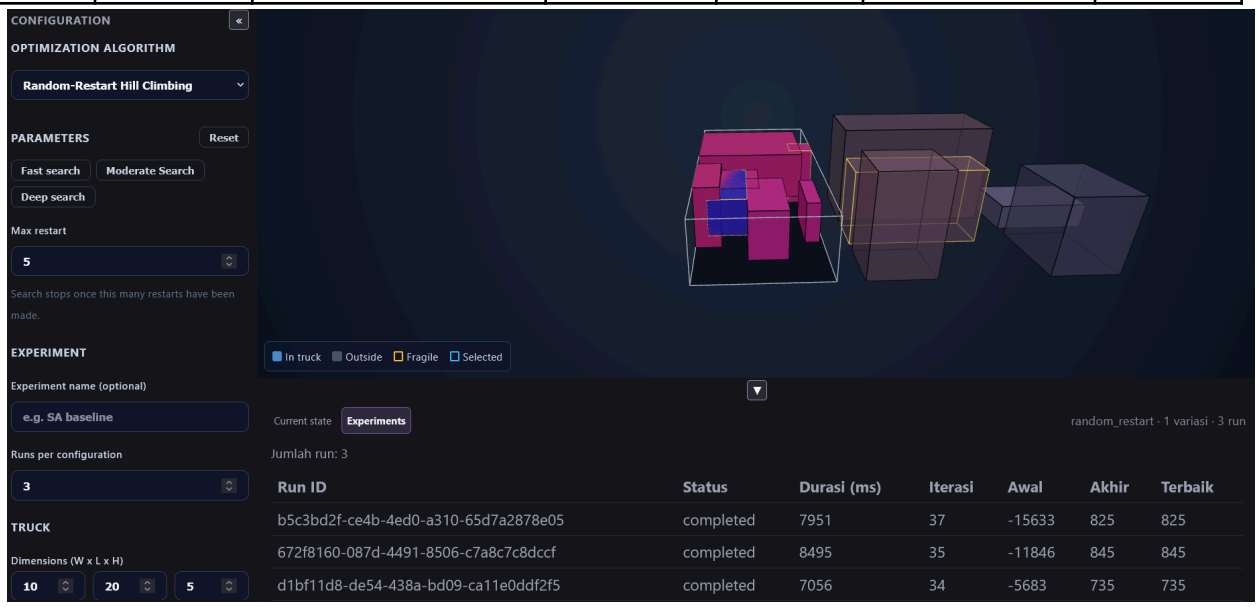
```
if (best_val is None or res_val > best_val):
                best = res
                best_val = res_val
            if (best_val >= goal):
                break
```

Pencarian berhenti lebih awal jika value solusi terbaik mencapai goal, yaitu jumlah nilai seluruh paket. Jika tujuan belum tercapai, pencarian berlanjut hingga batas restart terpenuhi.

Hasil Eksperimen:

Run	State awal	State akhir (paket di dalam truk)	Objective akhir	Banyak restart	Iterasi per restart (0-5)	Durasi
1	S1 (−15633)	PKG-1 (3,0,0) 6×3×4; PKG-2 (1,5,0) 8×4×5; PKG-3 (9,2,0) 1×3×4; PKG-6 (2,9,0) 5×2×2; PKG-8 (8,9,0)	**825**	5	5, 6, 5, 5, 5, 6 (total 32)	5057 ms

		2×5×5; PKG-9 (1,11,0) 7×9×4				
2	S2 (−11846)	PKG-2 (1,16,0) 8×4×5; PKG-3 (0,17,0) 1×3×4; PKG-6 (2,14,0) 5×2×2; PKG-7 (2,2,0) 8×12×5	**845**	5	3, 6, 6, 4, 8, 3 (total 30)	5421 ms
3	S3 (−5683)	PKG-1 (4,7,0) 6×3×4; PKG-2 (1,10,0) 4×8×5; PKG-3 (4,0,0) 3×1×4; PKG-5 (5,15,0) 4×4×4; PKG-6 (5,11,0) 2×2×5; PKG-11 (2,1,0) 6×6×5	**735**	5	4, 5, 4, 6, 5, 5 (total 29)	4718 ms



Analisis Eksperimen:

Random restart memberi hasil terbaik di antara semua algoritma (rata-rata 802, terbaik 845 atau 64% dari batas atas) dan cukup konsisten (selisih 110). Setiap pendakian sendiri sama terbatasnya dengan steepest ascent: berhenti setelah 3–8

iterasi di local optimum yang nilainya tersebar dari 460 sampai 845. Karena yang diambil adalah yang terbaik dari 6 sampel local optimum, peluang mendapat local optimum yang bagus jauh lebih besar.

VII. Simulated Annealing

Simulated Annealing memungkinkan perpindahan ke neighbor dengan value yang lebih rendah berdasarkan probabilitas tertentu. Semakin rendah temperatur, semakin kecil kemungkinan neighbor yang lebih buruk diterima. Dengan demikian, algoritma ini dapat mengeksplorasi solusi lain dan mengurangi risiko terjebak pada jebakan *local optimum*.

1. Konstruktor

```
def __init__(self, initial_temp: float = 1000.0, cooling_rate: float
= 0.99, min_temp: float = 0.01, rng=random):
        self.initial_temp = initial_temp
        self.cooling_rate = cooling_rate
        self.min_temp = min_temp
        self.rng = rng
        self.history: list[int] = []
        self.prob_history: list[float] = []
        self.stuck_frequency = 0
        self.duration = 0.0
```

Konstruktor mengatur temperatur awal, faktor penurunan temperatur, dan temperatur minimum. *History* mencatat value setiap iterasi, *prob_history* mencatat probabilitas penerimaan neighbor, *stuck_frequency* menghitung jumlah neighbor yang ditolak dan *duration* mencatat durasi pencarian

2. run(init_state, obj_fn)

```
curr = copy_state(init_state)
        curr_val = obj_fn(curr)

        best = copy_state(curr)
        best_val = curr_val

        self.history = [curr_val]
        self.prob_history = []
        self.stuck_frequency = 0
```

curr menyimpan state yang sedang dieksplorasi, sedangkan *best* menyimpan state dengan value tertinggi yang pernah ditemukan. Pemisahan keduanya memastikan algoritma mengembalikan solusi terbaik walau menerima perpindahan ke state dengan value lebih rendah

3. Pemilihan dan penerimaan neighbor

```
while temp > self.min_temp:
            neigh = random_neighbor(curr, self.rng)
            neigh_val = obj_fn(neigh)

            delta_e = neigh_val - curr_val

            if delta_e > 0:
                curr = neigh
                curr_val = neigh_val
                self.prob_history.append(1.0)
            else:
                prob = math.exp(delta_e / temp) if temp > 0 else 0
                self.prob_history.append(prob)

                if self.rng.random() < prob:
                    curr = neigh
                    curr_val = neigh_val
                else:
                    self.stuck_frequency += 1
```

Pada setiap iterasi, *random_neighbor()* menghasilkan satu secara acak. Jika value lebih tinggi, langsung diterima. Sebaliknya, jika valuenya sama atau lebih rendah, probabilitas penerimaan dihitung menggunakan rumus: Plot $e^{\frac{\Delta E}{T}}$.

4. Pencatatan hasil dan penurunan temperatur

```
self.history.append(curr_val)

            if curr_val > best_val:
                best = copy_state(curr)
                best_val = curr_val
```

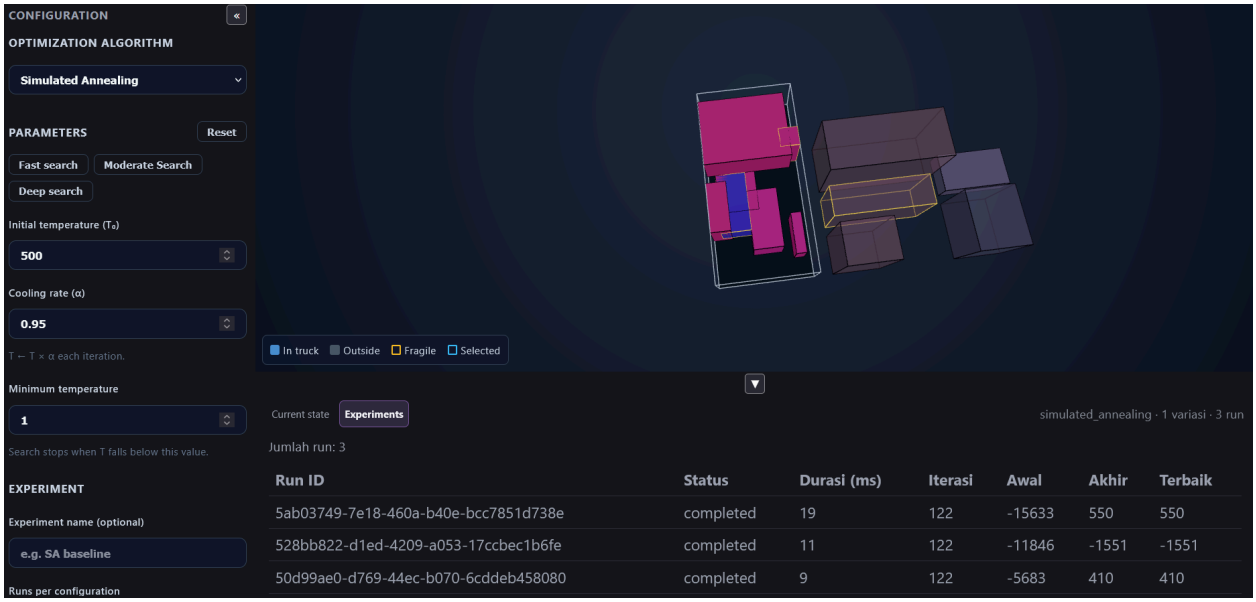
```
            temp *= self.cooling_rate
```

Value state saat ini dicatat pada setiap iterasi. *Best* akan diperbarui ketika solusi yang lebih baik ditemukan dari solusi terbaik sebelumnya. Temperatur kemudian diturunkan dengan mengalikannya dengan rate cooling hingga batas minimum.

Pencarian berhenti jika temperatur tidak lebih tinggi dari *min_temp*. Kemudian, durasi pencarian dicatat dan *best* dikembalikan sebagai solusi akhir.

Hasil Eksperimen:

Run	State awal	State akhir (paket di dalam truk)	Objective akhir	Iterasi	Frekuensi stuck	Durasi
1	S1 (−15633)	PKG-1 (3,8,0) 6×3×4; PKG-6 (8,13,0) 2×5×2; PKG-9 (0,11,0) 7×9×4; PKG-12 (1,2,0) 7×2×4	**550**	122	23 (18,9%)	7 ms
2	S2 (−11846)	PKG-2 (2,16,0) 8×4×5; PKG-5 (6,2,1) 4×4×4; PKG-6 (1,3,0) 5×2×2; PKG-8 (5,11,0) 5×5×2; PKG-12 (0,5,0) 4×7×2 *(masih 1 pelanggaran)*	**−1551**	122	48 (39,3%)	8 ms
3	S3 (−5683)	PKG-3 (2,19,0) 3×1×4; PKG-4 (4,6,0) 6×10×3; PKG-5 (2,1,0) 4×4×4	**410**	122	36 (29,5%)	8 ms



Analisis Eksperimen:

Di eksperimen ini SA berperilaku hampir seperti stochastic hill climbing. Penyebabnya skala objective yaitu satu pelanggaran bernilai −2051, sedangkan suhu awal hanya 500. Langkah yang menambah satu pelanggaran punya peluang diterima e^(−2051/500) ≈ 0,017 di awal, dan peluang itu cepat turun ke ≈0 saat T mengecil. Pada plot e^(ΔE/T), nilai hampir selalu 1 (neighbor lebih baik atau seri, yang selalu diterima) atau ≈0 (neighbor lebih buruk). Nilai di antaranya hanya muncul 6–8 kali per run, di iterasi awal dan untuk ΔE kecil (misalnya kehilangan satu paket bernilai kecil). Akibatnya dalam 122 iterasi hanya 2–4 langkah memburuk yang diterima, sehingga sifat eksplorasi SA hampir tidak muncul.

VIII. Genetic Algorithm

Genetic Algorithm menggunakan representasi state yang merupakan satu solusi lengkap berisi objek truck, packages, dan daftar representasi untuk tiap package. Implementasi Genetic Algorithm ada pada class GeneticAlgorithm.

1. Konstruktor

```
    def __init__(self, population_size: int, iterations: int,
mutation_rate: float = 0.1, rng=random):
        self.population_size = population_size
        self.iterations = iterations
        self.mutation_rate = mutation_rate
        self.rng = rng
```

```
        self.history: list[tuple[int, float]] = []
        self.duration = 0.0
```

Berisi atribut untuk mencatat jumlah populasi tiap iterasi, iterasi, peluang mutasi, random generator, serta durasi.

2. run(init_state, obj_fn)

```
        population = [copy_state(init_state)] + [
            random_state(init_state.truck, packages, self.rng) for _
in range(self.population_size - 1)
        ]
```

Populasi berisi salinan init_state ditambah population_size - 1 state acak. init_state diikutkan supaya hasil akhir tidak pernah lebih buruk dari state awal. Lalu dilakukan evaluasi fitness setiap individu, serta loop tiap generasi:

```
        fitness = [obj_fn(s) for s in population]
        best, best_fit = self.record(population, fitness)

        for _ in range(self.iterations):
            new_population = []
            for _ in range(self.population_size):
                x, y = self.random_selection(population, fitness)
                child = self.reproduce(x, y)
                if self.rng.random() < self.mutation_rate:
                    child = random_neighbor(child, self.rng)
                new_population.append(child)
            population = new_population
```

Di setiap generasi, anak dibuat sebanyak population_size. Untuk tiap child:

1. Seleksi: pilih dua induk x dan y
2. Crossover: gabungkan keduanya menjadi child
3. Mutasi: dengan peluang mutation_rate, child diganti dengan satu neighbor acaknya (satu kali swap, move, atau rotate).

Generasi lama diganti seluruhnya oleh anak-anaknya (generational replacement). Fitness dihitung ulang, statistik dicatat, dan best diperbarui kalau ada individu yang lebih baik dari semua yang pernah ditemukan.

3. random_selection(population, fitness)

```
        low = min(fitness)
        weights = [f - low + 1 for f in fitness]
        return self.rng.choices(population, weights=weights, k=2)
```

Memilih dua induk dengan roulette wheel selection: peluang terpilih sebanding dengan bobot, peluang_i = bobot_i / total_bobot. Bobot tidak boleh negatif, sedangkan score bisa negatif karena penalti.

4. reproduce(x, y)

```
        n = len(x.packages)
        c = self.rng.randint(1, n - 1) if n > 1 else 0
        child = copy_state(x)
        child.shape_pack_models[c:] =
copy_state(y).shape_pack_models[c:]
        return child
```

Penentuan point crossover ini terdiri atas :

- n: jumlah gen (paket).
- c: titik potong acak di [1, n-1], supaya anak selalu mendapat minimal satu gen dari masing-masing induk. Kalau hanya ada 1 paket, c = 0 dan anak menjadi salinan y.
- child = copy_state(x): anak dimulai sebagai salinan x.
- Gen dari indeks c sampai akhir diganti dengan gen y. copy_state(y) dipakai supaya anak tidak berbagi objek PackRepresentation dengan y; kalau berbagi, mutasi pada anak bisa ikut mengubah induknya.

Hasil Eksperimen:

Parameter tetap: mutation rate = 0,1.

1. Populasi sebagai kontrol (populasi = 20, variasi banyak iterasi 50, 100, 200)

Iterasi	Run	State awal	State akhir (paket di dalam truk)	Objective akhir	Generasi ditemukan	Durasi
50	1	S1	PKG-5 (2,5,0) 4×4×4; PKG-6 (0,3,0) 2×2×5; PKG-8 (4,2,0) 5×2×5	**205**	0	61 ms
50	2	S2	PKG-3 (1,13,0) 1×3×4; PKG-8 (0,5,0) 5×2×5;	**515**	46	63 ms

			PKG-9 (2,8,0) 7×9×4; PKG-12 (3,0,0) 7×2×4			
50	3	S3	PKG-2 (1,9,0) 8×4×5; PKG-8 (5,6,0) 5×2×5; PKG-12 (1,15,0) 7×4×2	**365**	7	64 ms
100	1	S1	PKG-5 (2,5,0) 4×4×4; PKG-6 (0,3,0) 2×2×5; PKG-8 (4,2,0) 5×2×5	**205**	0	115 ms
100	2	S2	PKG-3 (1,13,0) 1×3×4; PKG-8 (0,5,0) 5×2×5; PKG-9 (2,8,0) 7×9×4; PKG-12 (3,0,0) 7×2×4	**515**	46	145 ms
100	3	S3	PKG-2 (1,9,0) 8×4×5; PKG-8 (5,6,0) 5×2×5; PKG-12 (1,15,0) 7×4×2	**365**	7	126 ms
200	1	S1	PKG-3 (5,16,0) 4×3×1; PKG-5 (2,5,0) 4×4×4; PKG-6 (3,10,0) 5×2×2; PKG-8 (8,8,0) 2×5×5	**265**	166	238 ms
200	2	S2	PKG-3 (1,13,0) 1×3×4; PKG-8 (0,5,0) 5×2×5; PKG-9 (2,8,0) 7×9×4; PKG-12 (3,0,0) 7×2×4	**515**	46	258 ms
200	3	S3	PKG-2 (1,9,0) 8×4×5; PKG-8 (5,6,0) 5×2×5; PKG-12 (1,15,0) 7×4×2	**365**	7	261 ms

2. Iterasi sebagai kontrol

Populasi	Run	State awal	State akhir (paket di dalam truk)	Objective akhir	Generasi ditemukan	Durasi
10	1	S1	PKG-3 (0,3,0) 1×3×4; PKG-5 (2,5,0) 4×4×4; PKG-8 (4,2,0) 5×2×5	**220**	74	69 ms
10	2	S2	PKG-3 (1,13,0) 3×1×4; PKG-5 (0,16,0) 4×4×4; PKG-8 (1,7,0) 2×5×5	**220**	98	63 ms
10	3	S3	PKG-3 (6,7,0) 3×1×4; PKG-5 (3,16,0) 4×4×4; PKG-8 (7,14,0) 2×5×5; PKG-11 (0,3,0) 6×6×5	**420**	99	60 ms
20	1	S1	PKG-5 (2,5,0) 4×4×4; PKG-6 (0,3,0) 2×2×5; PKG-8 (4,2,0) 5×2×5	**205**	0	116 ms
20	2	S2	PKG-3 (1,13,0) 1×3×4; PKG-8 (0,5,0) 5×2×5; PKG-9 (2,8,0) 7×9×4; PKG-12 (3,0,0) 7×2×4	**515**	46	134 ms
20	3	S3	PKG-2 (1,9,0) 8×4×5; PKG-8 (5,6,0) 5×2×5; PKG-12 (1,15,0) 7×4×2	**365**	7	126 ms
40	1	S1	PKG-7 (1,6,0) 8×12×5; PKG-10 (0,3,0) 6×3×3	**600**	99	245 ms
40	2	S2	PKG-2 (1,15,0) 8×4×5; PKG-9 (0,0,0) 9×7×4; PKG-11 (0,9,0) 6×6×5	**730**	1	253 ms
40	3	S3	PKG-5 (5,2,0) 4×4×4; PKG-6 (4,9,0) 2×2×5; PKG-8 (7,7,0) 2×5×5;	**360**	8	262 ms

			PKG-10 (2,0,0) 3×6×3; PKG-12 (1,15,0) 7×4×2			

Analisis Eksperimen:

Pengaruh banyak iterasi. Dengan populasi tetap 20, menambah iterasi hampir tidak mengubah hasil (361,7→361,7→381,7). Seed yang sama memakai urutan bilangan acak yang sama, sehingga run dengan iterasi lebih banyak adalah kelanjutan dari run yang lebih pendek. Masalahnya, individu terbaik di run 1, 2, dan 3 sudah ditemukan dan generasi-generasi berikutnya tidak pernah melewatinya. Satu-satunya perbaikan adalah run 1 dengan 200 iterasi (205→265). Durasi naik linear terhadap iterasi (63→129→252 ms), karena biaya per generasi tetap (populasi × evaluasi objective). Jadi pada populasi 20, tambahan iterasi sebagian besar terbuang.

Pengaruh jumlah populasi. Dengan iterasi tetap 100, menaikkan populasi menaikkan hasil dengan jelas (287→362→ 563). Populasi 10 selalu menghasilkan state valid, tetapi isinya hanya 3–4 paket kecil, dan individu terbaiknya baru ditemukan di generasi 74–99, yang menandakan pencarian sangat lambat. Populasi kecil membuat keragaman genetik cepat habis, sehingga crossover hanya menggabungkan individu yang mirip dan pencarian bergantung pada mutasi. Populasi 40 menghasilkan dua nilai GA tertinggi: 730 (berisi PKG-2, PKG-9, PKG-11) dan 600 (berisi PKG-7). Populasi lebih besar berarti lebih banyak state acak di generasi awal dan lebih banyak kombinasi crossover, sehingga peluang menemukan state dengan paket besar meningkat. Durasi naik linear terhadap populasi (64→125→253 ms).

Kesimpulan dan Saran

1. Kesimpulan

a. Hill-Climbing: Steepest Ascent cepat mencapai local optimum, Sideaways membantu mengeksplorasi plateau, Stochastic paling cepat tetapi kualitas solusinya kurang konsisten, sedangkan Random Restart menghasilkan solusi terbaik di antara varian yang diuji
b. Simulated Annealing: Belum optimal, terutama karena temperatur awal lebih kecil dibandingkan penalti pelanggaran, sehingga dalam menerima peluang solusi yang lebih buruk itu sangat rendah
c. Genetic Algorithm: Penambahan iterasi tidak selalu meningkatkan kualitas solusi. Peningkatan populasi berpotensi menghasilkan solusi lebih baik, tetapi masih bergantung ke seed dan parameter
d. Dalam membandingkan algoritma - algoritma yang telah duij, Random Restart memperoleh nilai objektif tertinggi sebesar 845, sedangkan Hill-Climbing Stochastic memiliki waktu eksekusi tercepat, sekitar 6-7 ms pada eksperimen

2. Saran

a. Dalam mengoptimasi Simulated Annealing, temperatur awal dan cooling rate harus lebih sesuai dengan skala penalti fungsi objektif
b. Dalam meningkatkan eksplorasi pada Hill-Climbing, batas sideways move dan jumlah restart dapat diuji dengan beberapa nilai berbeda. Selain itu mekanisme pemilihan tetangga dapat dikembangkan agar pencarian lebih berpeluang menemukan konfigurasi yang membutuhkan beberapa perubahan posisi sebelum memberikan peningkatan nilai objektif
c. Tiga run memberikan gambaran awal, tetapi belum cukup untuk menyimpulkan performa umum setiap algoritma. Diperlukan pengujian dengan jumlah yang lebih banyak untuk mendapatkan gambaran perbandingan yang lebih jelas

Pembagian Tugas

Nama	NIM	Pembagian Tugas
Mikhael Benrael Tampubolon	13524009	Simulated Annealing, Bonus Objective, laporan, backend
Marcel Luther Sitorus	13524063	Hill-Climbing, Visualisasi, Frontend,
Richard Samuel Simanullang	13524112	Genetic Algorithm, Bonus varian Hill-Climbing, backend

Referensi

[1] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. Available: [URL]. [Accessed: Oct. 10, 2026].

[2] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Oct. 10, 2026].

[3] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Oct. 10, 2026].

[4] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: [URL]. [Accessed: Oct. 10, 2026].

[5] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: [URL]. [Accessed: Oct. 10, 2026].

Lampiran

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM:

- Mikhael Benrael Tampubolon - 13524009
- Marcel Luther Sitorus - 13524063
- Richard Samuel Simanullang - 13524112

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi

Kode Mata Kuliah : IF3170

Nama Mata Kuliah : Inteligensi Artifisial

Judul Tugas/Proposal : Tugas Besar I - Implementasi Penyusunan Paket Container Menggunakan Local Search

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika ya, alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, DeepL, QuillBot, GrammarBot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	
Bantuan Diskusi dalam Penyusunan	Ya	2 – AI-assisted idea

Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom Companion, atau sejenisnya.		generation and structuring
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	